COMP2007: Operating Systems & Concurrency
Week 8 – 12:00pm Wednesday – 13 November 2024

valid for 75 minutes from 11:45am
generated 2024-09-20 14:34

Figure: Attendance Monitoring

Operating Systems and Concurrency

Memory Management 5

COMP2007

Geert De Maere
(Dan Marsden)
{Geert.DeMaere,Dan.Marsden}@Nottingham.ac.uk

University Of Nottingham
United Kingdom

2024

Announcements

- Some minor clarifications have been added to the coursework specification
- EoI Feedback will be released today (CVs and marks to follow)

Goals for Today

Overview

- **Page tables**: **inverted page tables** and **paging performance** (**multi-level page tables**)
- **Key design decisions** when using **virtual memory**
  - **Page fetching**: demand paging, pre-paging
  - **Page replacement**: what pages to **removed** from memory
- Page **replacement policies**

Recall

Last Lecture

- **Virtual memory** relies on **localities** that constitute **groups of pages** which are **used together**, e.g., related to a function (code, data, etc.)
  - Processes **move between localities**
  - If all required pages are **in memory**, **no page faults** will be generated
- **Page tables** become **larger** and **more complex** (present/absent bits, referenced/modified bits)

Inverted Page Tables

Concept

- A **“normal” page table’s size** is proportional to the number of pages in the virtual address space (significant for 64-bit machines)
- An **“inverted” page table’s** size is proportional to the size of main memory
  - Contains one **entry for every frame** (i.e. not for every page)
  - Is **indexed by frame number** / **hash code** (not by page number)
  - When referencing memory, the OS must **search the inverted page ta** rresponding entry (i.e. page and process id) $\Rightarrow$ this could be too slow.
- **Inverted page tables** use a **hash function** that transforms page nu s) and PID into an index in the inverted page table (hash table)

Inverted Page Tables

Page Table Entries

- Process Identifier (**PID**): the process that owns this page.
- Page Number (logical address space)
- **Protection** bits (Read/Write/Execution)
- **Link Pointer**: field points toward the **next entry** (hash table can have collisions)

PID	Page Number	RWX	Link Pointer	Frame Number

Figure: Example of an Inverted Page Table Entry (other info bits are not shown here)

Inverted Page Tables

Address translation

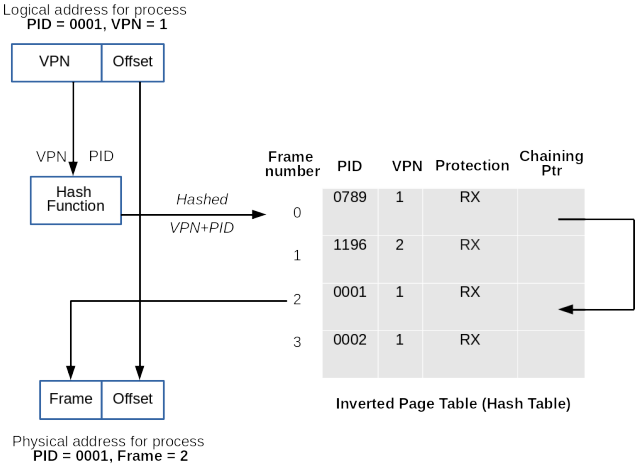

Figure: Address Translation with an Inverted Page Table

Inverted Page Tables

Address translation

Questions

1. Why is the **PID required** in the entry for an inverted page table?
2. Why is $page_number \% number_of_frames = hash_index$ not a good approach?

Virtual Memory

Page Tables Revisited: Inverted Page Tables

- Advantages:
  - The OS maintains a **single inverted page table** for all processes
  - **Space saving**, especially for large virtual address spaces
- Disadvantages:
  - Logical-to-physical address translation becomes **harder** / **slower**
  - **Collisions** have to be handled and will slow down address translation:
    - Hash tables reduce search time
    - TLBs help to improve performance
- Used on some 64-bit machines and OSs
  `http://answers.microsoft.com/en-us/windows/forum/windows_10-performance/physical-and-virtual-memory-in-windows-10/e36fb5bc-9ac8-49af-951c-e7d39b979938`

Paging Decisions

Fetching and Replacement

- **Two key decisions** have to be made using virtual memory
  - What pages are **loaded** and when ⇒ **page fetching policies**
  - What pages are **removed** from memory and when ⇒ **page replacement algorithms**
- **Pages are shuttled** between primary and secondary memory

Page Fetching

Demand Paging

- **Demand paging** starts the process with **no pages in memory**
  - The first instruction will immediately cause **a page fault**
  - **More page faults** will follow, but they will **stabilise over time** until moving to the **next locality**
  - The set of pages that is currently being **used** is called its **working set** ($\Leftrightarrow$ resident set)
- Pages are only **loaded when needed**, i.e. following **page faults**

Page Fetching

Pre-paging

- When the process is (re-)started, all pages expected to be used (i.e. the working set) c **brought into memory at once**
  - This can drastically **reduce the page fault rate**
  - Retrieving multiple (**contiguously stored**) pages **reduces transfer times** (seek time, rotational latency, etc.)
- **Pre-paging** loads pages (as much as possible) **before page faults are generated** ($\Rightarrow$ a similar mechanism is used when processes are **swapped out** / **in**)

Page Fetching

Pre-Paging

- Keep track of the **working set** to enable loading pages when restarting a process
- Aims to **reduce page faults**
- One has to track **which pages make up the working set**
  - Use an **aging algorithm** (e.g. reference bits that are shifted)

Page Fetching

Memory Access Times and Performance

- Avoiding **unnecessary pages** and **page replacement** is important!
- Let:
  - ma denote the **memory access time** (multiple times for multi-level page table)
  - p denote the **page fault rate** $\Rightarrow 1-p$ is the **"hit rate"**
  - pft denote the **page fault time**
- The **effective access time** is then given by:

$$T_a = (1-p) \times ma + pft \times p \tag{1}$$

- Note that we are not considering here TLBs (embedded in the average value of ma)

Page Fetching

Memory Access Times and Performance

- For a single-level page table:
  - Memory **access time** of 100ns (10^{-9})
  - Two memory accesses required (200ns)
  - A **page fault time** of 8ms (10^{-3}, recall that hard drives are slow)

$$T_a = (1 - p) \times 200 + p \times 8000000 \tag{2}$$

- The effective access time is **proportional to page fault rate**
- **Avoiding page faults** is critical

Page Replacement

Concepts

- The OS must choose a **page to remove** when a new **one is loaded** (and all are occupied)
- This choice is made by **page replacement algorithms** and **takes into account**
  - When the page was **last used** / **expected to be used** again
  - Whether the **page has been modified** (only modified pages need to be written)
- Replacement decisions must be **made intelligently** (in most cases) to **save time**

Page Replacement

Algorithms

1. **Optimal** page replacement
2. **FIFO** page replacement
   - Second chance replacement
   - Clock replacement
3. **Not recently used** (NRU)
4. **Least recently used** (LRU)

Page Replacement

Optimal Page Replacement

- In an **ideal** / **optimal** world
  - Every page is labeled with the **number of instructions** that will be executed / length of time before it is **used again**
  - The page which **will not be referenced** for the **longest time** is the optimal one to remove

Optimal Page Replacement

This is perfect! Right?

Page Replacement

Optimal Page Replacement

- The **optimal approach** is **infeasible** in practice
  - It can be used for **post-execution analysis**
  - It provides a **lowerbound** on the **number of page faults**
  - Used for comparison with other algorithms

Page Replacement

First-In, First-Out (FIFO)

- FIFO maintains a **linked list** and **new pages** are added at the end of the list
- The **oldest page** at the **head of the list** is evicted when a page fault occurs
- The **(dis-)advantages** of FIFO include:
  - It is **easy** to understand/implement
  - It **performs poorly** $\Rightarrow$ heavily used pages are just as likely to be evicted as a lightly used pages

Page Replacement

FIFO Simulation

- Assume we have a system with **eight logical pages** and **four physical frames** (PFs)
- Consider the following page **references in order**:

  0 2 1 3 5 4 6 3 7 4 7 3 3 5 5 3 1 1 1 7 2 3 1 4

- The number of **page faults** that are generated is **13**

	0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	2	3	1	4
PF1																								
PF2																								
PF3																								
PF4																								

Figure: FIFO Page Replacement

©University of Nottingham

Page Replacement

FIFO Simulation

- Assume we have a system with **eight logical pages** and **four physical frames** (PFs)
- Consider the following page **references in order**:

  0 2 1 3 5 4 6 3 7 4 7 3 3 5 5 3 1 1 1 7 2 3 1 4

- The number of **page faults** that are generated is **13**

	0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	2	3	1	4
PF1	0																							
PF2	-																							
PF3	-																							
PF4	-																							

Figure: FIFO Page Replacement

©University of Nottingham

Page Replacement

FIFO Simulation

- Assume we have a system with **eight logical pages** and **four physical frames** (PFs)
- Consider the following page **references in order**:

  0 2 1 3 5 4 6 3 7 4 7 3 3 5 5 3 1 1 1 7 2 3 1 4

- The number of **page faults** that are generated is **13**

	0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	2	3	1	4
PF1	0	0																						
PF2	-	2																						
PF3	-	-																						
PF4	-	-																						

Figure: FIFO Page Replacement

©University of Nottingham

Page Replacement

FIFO Simulation

- Assume we have a system with **eight logical pages** and **four physical frames** (PFs)
- Consider the following page **references in order**:

  0 2 1 3 5 4 6 3 7 4 7 3 3 5 5 3 1 1 1 7 2 3 1 4

- The number of **page faults** that are generated is **13**

	0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	2	3	1	4
PF1	0	0	0																					
PF2	-	2	2																					
PF3	-	-	1																					
PF4	-	-	-																					

Figure: FIFO Page Replacement

©University of Nottingham

Page Replacement

FIFO Simulation

- Assume we have a system with **eight logical pages** and **four physical frames** (PFs)
- Consider the following page **references in order**:

  0 2 1 3 5 4 6 3 7 4 7 3 3 5 5 3 1 1 1 7 2 3 1 4

- The number of **page faults** that are generated is **13**

	0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	2	3	1	4
PF1	0	0	0	0																				
PF2	-	2	2	2																				
PF3	-	-	1	1																				
PF4	-	-	-	3																				

Figure: FIFO Page Replacement

©University of Nottingham

Page Replacement

FIFO Simulation

- Assume we have a system with **eight logical pages** and **four physical frames** (PFs)
- Consider the following page **references in order**:

  0 2 1 3 5 4 6 3 7 4 7 3 3 5 5 3 1 1 1 7 2 3 1 4

- The number of **page faults** that are generated is **13**

	0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	2	3	1	4
PF1	0	0	0	0	5																			
PF2	-	2	2	2	2																			
PF3	-	-	1	1	1																			
PF4	-	-	-	3	3																			

Figure: FIFO Page Replacement

©University of Nottingham

Page Replacement

FIFO Simulation

- Assume we have a system with **eight logical pages** and **four physical frames** (PFs)
- Consider the following page **references in order**:

  0 2 1 3 5 4 6 3 7 4 7 3 3 5 5 3 1 1 1 7 2 3 1 4

- The number of **page faults** that are generated is **13**

	0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	2	3	1	4
PF1	0	0	0	0	5	5																		
PF2	-	2	2	2	2	4																		
PF3	-	-	1	1	1	1																		
PF4	-	-	-	3	3	3																		

Figure: FIFO Page Replacement

©University of Nottingham

Page Replacement

FIFO Simulation

- Assume we have a system with **eight logical pages** and **four physical frames** (PFs)
- Consider the following page **references in order**:

  0 2 1 3 5 4 6 3 7 4 7 3 3 5 5 3 1 1 1 7 2 3 1 4

- The number of **page faults** that are generated is **13**

	0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	2	3	1	4
PF1	0	0	0	0	5	5	5																	
PF2	-	2	2	2	2	4	4																	
PF3	-	-	1	1	1	1	6																	
PF4	-	-	-	3	3	3	3																	

Figure: FIFO Page Replacement

©University of Nottingham

Page Replacement

FIFO Simulation

- Assume we have a system with **eight logical pages** and **four physical frames** (PFs)
- Consider the following page **references in order**:

  0 2 1 3 5 4 6 3 7 4 7 3 3 5 5 3 1 1 1 7 2 3 1 4

- The number of **page faults** that are generated is **13**

	0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	2	3	1	4
PF1	0	0	0	0	5	5	5	5																
PF2	-	2	2	2	2	4	4	4																
PF3	-	-	1	1	1	1	6	6																
PF4	-	-	-	3	3	3	3	3																

Figure: FIFO Page Replacement

Page Replacement

FIFO Simulation

- Assume we have a system with **eight logical pages** and **four physical frames** (PFs)
- Consider the following page **references in order**:

  0 2 1 3 5 4 6 3 7 4 7 3 3 5 5 3 1 1 1 7 2 3 1 4

- The number of **page faults** that are generated is **13**

	0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	2	3	1	4
PF1	0	0	0	0	5	5	5	5	5															
PF2	-	2	2	2	2	4	4	4	4															
PF3	-	-	1	1	1	1	6	6	6															
PF4	-	-	-	3	3	3	3	3	7															

Figure: FIFO Page Replacement

©University of Nottingham

Page Replacement

FIFO Simulation

- Assume we have a system with **eight logical pages** and **four physical frames** (PFs)
- Consider the following page **references in order**:

  0 2 1 3 5 4 6 3 7 4 7 3 3 5 5 3 1 1 1 7 2 3 1 4

- The number of **page faults** that are generated is **13**

	0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	2	3	1	4
PF1	0	0	0	0	5	5	5	5	5	5	5													
PF2	-	2	2	2	2	4	4	4	4	4	4													
PF3	-	-	1	1	1	1	6	6	6	6	6													
PF4	-	-	-	3	3	3	3	3	7	7	7													

Figure: FIFO Page Replacement

©University of Nottingham

Page Replacement

FIFO Simulation

- Assume we have a system with **eight logical pages** and **four physical frames** (PFs)
- Consider the following page **references in order**:

  0 2 1 3 5 4 6 3 7 4 7 3 3 5 5 3 1 1 1 7 2 3 1 4

- The number of **page faults** that are generated is **13**

	0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	2	3	1	4
PF1	0	0	0	0	5	5	5	5	5	5	5	3												
PF2	-	2	2	2	2	4	4	4	4	4	4	4												
PF3	-	-	1	1	1	1	6	6	6	6	6	6												
PF4	-	-	-	3	3	3	3	3	7	7	7	7												

Figure: FIFO Page Replacement

©University of Nottingham

Page Replacement

FIFO Simulation

- Assume we have a system with **eight logical pages** and **four physical frames** (PFs)
- Consider the following page **references in order**:

  0 2 1 3 5 4 6 3 7 4 7 3 3 5 5 3 1 1 1 7 2 3 1 4

- The number of **page faults** that are generated is **13**

	0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	2	3	1	4
PF1	0	0	0	0	5	5	5	5	5	5	5	3	3											
PF2	-	2	2	2	2	4	4	4	4	4	4	4	4											
PF3	-	-	1	1	1	1	6	6	6	6	6	6	6											
PF4	-	-	-	3	3	3	3	3	7	7	7	7	7											

Figure: FIFO Page Replacement

©University of Nottingham

Page Replacement

FIFO Simulation

- Assume we have a system with **eight logical pages** and **four physical frames** (PFs)
- Consider the following page **references in order**:

  0 2 1 3 5 4 6 3 7 4 7 3 3 5 5 3 1 1 1 7 2 3 1 4

- The number of **page faults** that are generated is **13**

	0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	2	3	1	4
PF1	0	0	0	0	5	5	5	5	5	5	5	3	3	3										
PF2	-	2	2	2	2	4	4	4	4	4	4	4	4	5										
PF3	-	-	1	1	1	1	6	6	6	6	6	6	6	6										
PF4	-	-	-	3	3	3	3	3	7	7	7	7	7	7										

Figure: FIFO Page Replacement

©University of Nottingham

Page Replacement

FIFO Simulation

- Assume we have a system with **eight logical pages** and **four physical frames** (PFs)
- Consider the following page **references in order**:

  0 2 1 3 5 4 6 3 7 4 7 3 3 5 5 3 1 1 1 7 2 3 1 4

- The number of **page faults** that are generated is **13**

	0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	2	3	1	4
PF1	0	0	0	0	5	5	5	5	5	5	5	3	3	3	3	3								
PF2	-	2	2	2	2	4	4	4	4	4	4	4	4	5	5	5								
PF3	-	-	1	1	1	1	6	6	6	6	6	6	6	6	6	6								
PF4	-	-	-	3	3	3	3	3	7	7	7	7	7	7	7	7								

Figure: FIFO Page Replacement

©University of Nottingham

Page Replacement

FIFO Simulation

- Assume we have a system with **eight logical pages** and **four physical frames** (PFs)
- Consider the following page **references in order**:

  0 2 1 3 5 4 6 3 7 4 7 3 3 5 5 3 1 1 1 7 2 3 1 4

- The number of **page faults** that are generated is **13**

	0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	2	3	1	4
PF1	0	0	0	0	5	5	5	5	5	5	5	3	3	3	3	3	3							
PF2	-	2	2	2	2	4	4	4	4	4	4	4	4	5	5	5	5							
PF3	-	-	1	1	1	1	6	6	6	6	6	6	6	6	6	6	1							
PF4	-	-	-	3	3	3	3	3	7	7	7	7	7	7	7	7	7							

Figure: FIFO Page Replacement

©University of Nottingham

Page Replacement

FIFO Simulation

- Assume we have a system with **eight logical pages** and **four physical frames** (PFs)
- Consider the following page **references in order**:

  0 2 1 3 5 4 6 3 7 4 7 3 3 5 5 3 1 1 1 7 2 3 1 4

- The number of **page faults** that are generated is **13**

	0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	2	3	1	4
PF1	0	0	0	0	5	5	5	5	5	5	5	3	3	3	3	3	3	3	3	3				
PF2	-	2	2	2	2	4	4	4	4	4	4	4	4	5	5	5	5	5	5	5				
PF3	-	-	1	1	1	1	6	6	6	6	6	6	6	6	6	6	1	1	1	1				
PF4	-	-	-	3	3	3	3	3	7	7	7	7	7	7	7	7	7	7	7	7				

Figure: FIFO Page Replacement

©University of Nottingham

Page Replacement

FIFO Simulation

- Assume we have a system with **eight logical pages** and **four physical frames** (PFs)
- Consider the following page **references in order**:

  0 2 1 3 5 4 6 3 7 4 7 3 3 5 5 3 1 1 1 7 2 3 1 4

- The number of **page faults** that are generated is **13**

	0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	2	3	1	4
PF1	0	0	0	0	5	5	5	5	5	5	5	3	3	3	3	3	3	3	3	3	3			
PF2	-	2	2	2	2	4	4	4	4	4	4	4	4	5	5	5	5	5	5	5	5			
PF3	-	-	1	1	1	1	6	6	6	6	6	6	6	6	6	6	1	1	1	1	1			
PF4	-	-	-	3	3	3	3	3	7	7	7	7	7	7	7	7	7	7	7	7	2			

Figure: FIFO Page Replacement

©University of Nottingham

Page Replacement

FIFO Simulation

- Assume we have a system with **eight logical pages** and **four physical frames** (PFs)
- Consider the following page **references in order**:

  0 2 1 3 5 4 6 3 7 4 7 3 3 5 5 3 1 1 1 7 2 3 1 4

- The number of **page faults** that are generated is **13**

	0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	2	3	1	4
PF1	0	0	0	0	5	5	5	5	5	5	5	3	3	3	3	3	3	3	3	3	3	3	3	
PF2	-	2	2	2	2	4	4	4	4	4	4	4	4	5	5	5	5	5	5	5	5	5	5	
PF3	-	-	1	1	1	1	6	6	6	6	6	6	6	6	6	6	1	1	1	1	1	1	1	
PF4	-	-	-	3	3	3	3	3	7	7	7	7	7	7	7	7	7	7	7	7	2	2	2	

Figure: FIFO Page Replacement

©University of Nottingham

Page Replacement

FIFO Simulation

- Assume we have a system with **eight logical pages** and **four physical frames** (PFs)
- Consider the following page **references in order**:

  0 2 1 3 5 4 6 3 7 4 7 3 3 5 5 3 1 1 1 7 2 3 1 4

- The number of **page faults** that are generated is **13**

	0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	2	3	1	4
PF1	0	0	0	0	5	5	5	5	5	5	5	3	3	3	3	3	3	3	3	3	3	3	3	4
PF2	-	2	2	2	2	4	4	4	4	4	4	4	4	5	5	5	5	5	5	5	5	5	5	5
PF3	-	-	1	1	1	1	6	6	6	6	6	6	6	6	6	6	1	1	1	1	1	1	1	1
PF4	-	-	-	3	3	3	3	3	7	7	7	7	7	7	7	7	7	7	7	7	2	2	2	2

Figure: FIFO Page Replacement

©University of Nottingham

Page Replacement

First-In, First-Out (FIFO)

First-In, First-Out (FIFO)

- The **oldest page** in memory is not necessarily the "**least frequently used**" one
- How would you account for this?

Page Replacement

Second Chance FIFO

- Second chance **FIFO**:
  - If a page at the front of the list has **not been referenced** it is **evicted**
  - If the reference bit is set, the page is **placed at the end** of list and its **reference bit reset**
- Second chance FIFO **works better** than FIFO, but and **can degrade to FIFO** if all pages were initially referenced

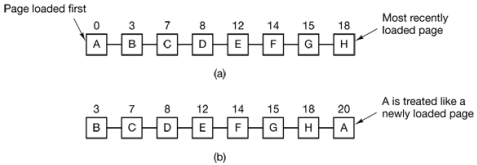

Figure: Second-Chance Algorithm (Tanenbaum)

Page Replacement

Second Chance FIFO

Second Chance FIFO

- Second Chance FIFO is **costly to implement** (list changes constantly)
- How would you resolve this?

Page Replacement

The Clock Replacement Algorithm

- **Second-chance FIFO** can be improved by **maintaining a circular list**
  - A **pointer** points to the last page"visited"
  - The algorithm is called (one-handed) **clock**
  - It is **slow for long lists**
- The **time spent** on maintaining the list **is reduced**

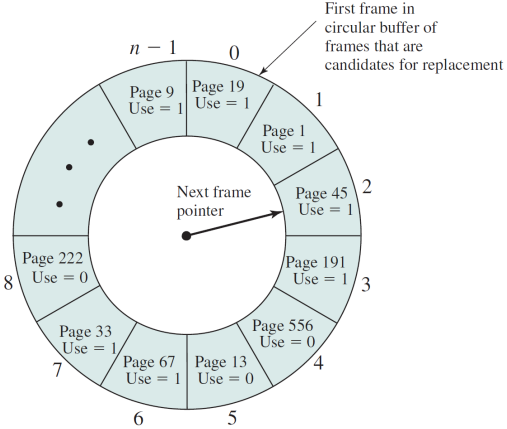

Figure: Clock Replacement Algorithm (Stallings)

Page Replacement

The Clock Replacement Algorithm

The Clock Replacement Algorithm

Would you implement this as a **circular linked list**, or an **linear array**?

Recap

Take-Home Message

- **Inverted** page tables
- **Fetching policies** (demand paging, pre-paging) and **access times**
- **Page replacement strategies** (optimal, FIFO)